Threat Intelligence & Automated Incident Response Lab

Live honeypot deployment and LLM-driven triage — T-Pot, Splunk, Canarytokens, and automated MITRE ATT&CK classification with the Anthropic API.

Tahmid Abtahee · github.com/tahosprojects · linkedin.com/in/tahmidabtahee · July 2026

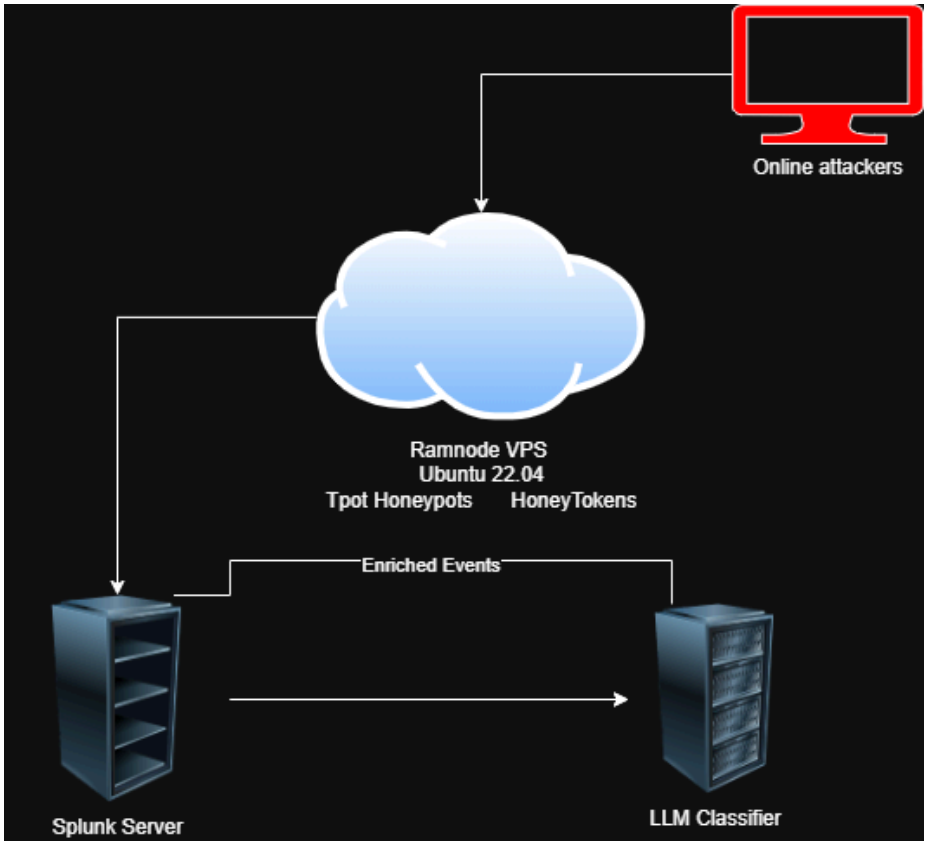
Overview

My earlier labs worked against attacks I generated myself — Active Directory and endpoint telemetry on-premises, then cloud attack emulation and detection engineering in AWS. This project removes the safety net. I built it to work with real attacks: a live honeypot exposed to the open internet, capturing genuine attacker behavior, with the goal of getting as close as possible to the day-to-day work of the field I am entering.

The result is an end-to-end pipeline that mirrors, at small scale, what a SOC or MDR provider does. Internet attackers hit a suite of honeypots on a public VPS, the telemetry flows into a self-hosted Splunk instance, honeytokens sit on the host as a deception tripwire, and a Python pipeline pulls captured events from Splunk, classifies each one against MITRE ATT&CK using the Anthropic API, and writes the enriched, analyst-ready results back into the index.

Stack: T-Pot 24.04.1 (20+ Docker honeypots), Splunk Enterprise, Canarytokens, Python (Splunk REST API, HTTP Event Collector, Anthropic API), RamNode KVM VPS on Ubuntu 22.04 — administered entirely over SSH.

Architecture



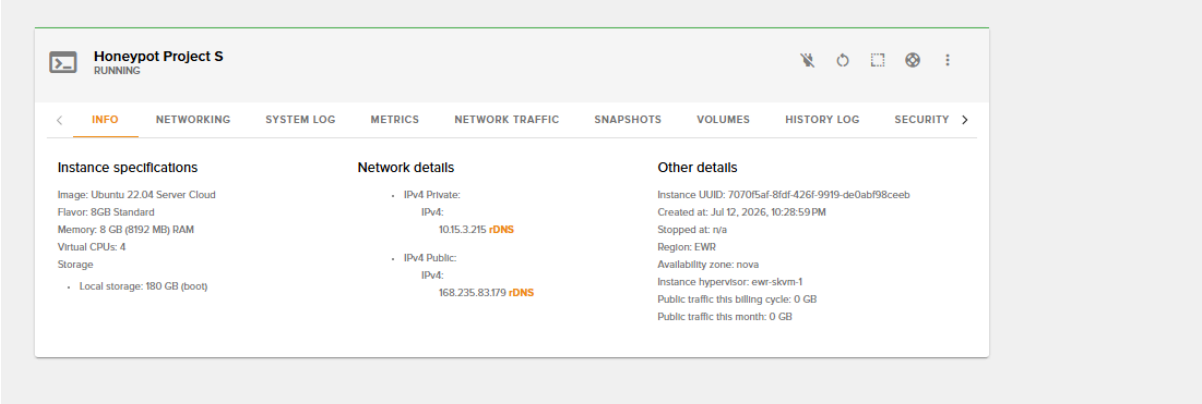
End-to-end data flow: internet attackers → T-Pot honeypots + honeytokens on the VPS → Splunk → LLM classifier → enriched events written back into Splunk.

Attackers reach the honeypot services exposed on a single cloud VPS. T-Pot's containerized honeypots and the planted honeytokens both live on that instance. Splunk, running on the same box, ingests the honeypot logs and acts as the system of record. The classifier closes the loop: it reads recent events from Splunk over the REST API, maps them to ATT&CK, and writes the enriched events back in over the HTTP Event Collector under their own sourcetype.

Everything runs on one VPS by design. Keeping Splunk beside the honeypots eliminated the networking and tunneling complexity of shipping logs to a second machine, at the cost of managing resource contention between the services on a single host — a trade-off that shaped much of the build.

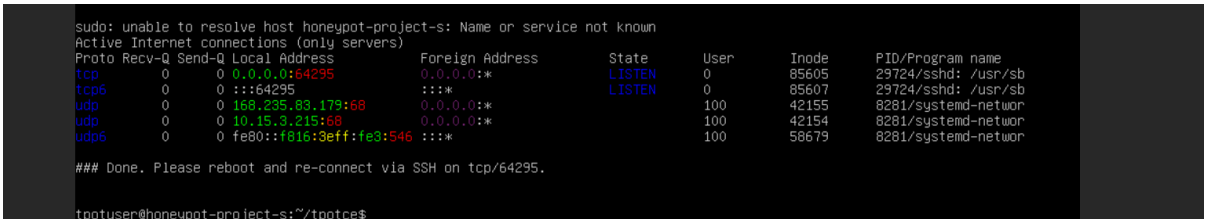
Infrastructure (RamNode VPS)

The lab runs on a RamNode KVM VPS in the EWR (New Jersey) region on Ubuntu 22.04 Server Cloud, provisioned initially at 8GB RAM and 4 vCPUs to keep cost down while validating the install. Every part of the build — Splunk, T-Pot, the classifier — was installed and administered remotely over SSH from my own machine; there is no console workflow anywhere in this project.



Initial instance specifications — 8GB RAM, 4 vCPUs, 180GB storage, Ubuntu 22.04, region EWR.

T-Pot's installer relocates the real SSH daemon from port 22 to 64295 as part of its hardening, because the honeypot suite deliberately occupies the common service ports — including 22 — as bait. After the installer finished, I confirmed with ss -tlnp that sshd was listening only on the new port before reconnecting.



Verifying sshd listening on port 64295 after the T-Pot installer relocated the service.

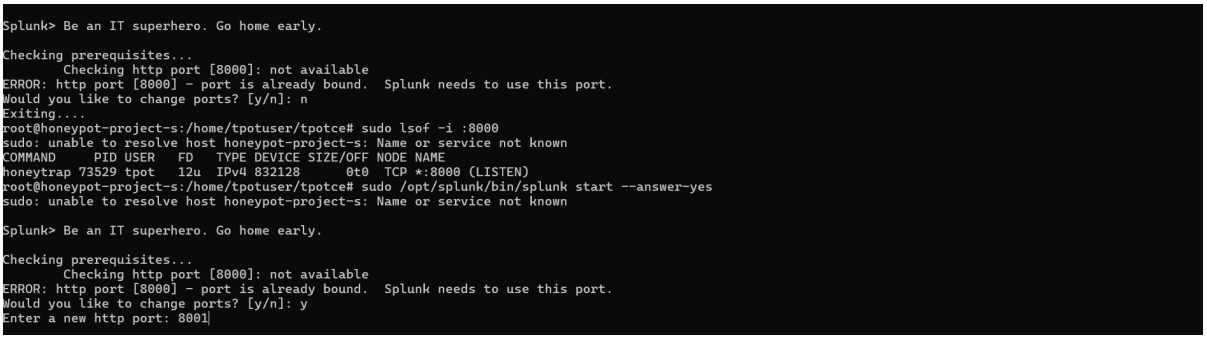
Worth stating explicitly: moving SSH to a high port reduces scanner noise, but it is obscurity, not a control. The habit that actually matters when connecting to a freshly provisioned VPS — or reconnecting after a port change like this one — is verifying the SSH host key fingerprint before accepting it. An unexpected host key change on a machine you have connected to before is one of the few reliable indicators of a man-in-the-middle condition, and the check costs nothing.

A note on the network outage. Shortly after provisioning, the instance went completely unreachable — SSH, web, even ICMP — despite correct server-side configuration. I ran MTR from multiple independent networks to rule out my own ISP, then comparison-tested against a second RamNode IP in the same region to isolate the fault to this instance's routing specifically. With that evidence attached, I opened a support ticket; RamNode confirmed and fixed a network-side issue on their end. Escalate to a vendor only after eliminating every variable you control — and bring evidence.

SIEM Installation (Splunk)

I began the software build by installing Splunk Enterprise directly on the instance over SSH. Splunk was chosen as the system of record over T-Pot's bundled Elastic stack because it matches the tooling from my previous detection engineering lab and what I am most likely to run in a SOC role.

Splunk's default web port (8000) immediately collided with Honeytrap, T-Pot's catch-all honeypot, which binds broadly across common ports by design. The first startup attempt failed with the port already bound; lsof -i :8000 confirmed Honeytrap was holding it. Rather than fight the honeypot for ports that are deliberately part of its attack surface, I relocated Splunk's web UI — first to 8001, and after further collisions, permanently to 8890.



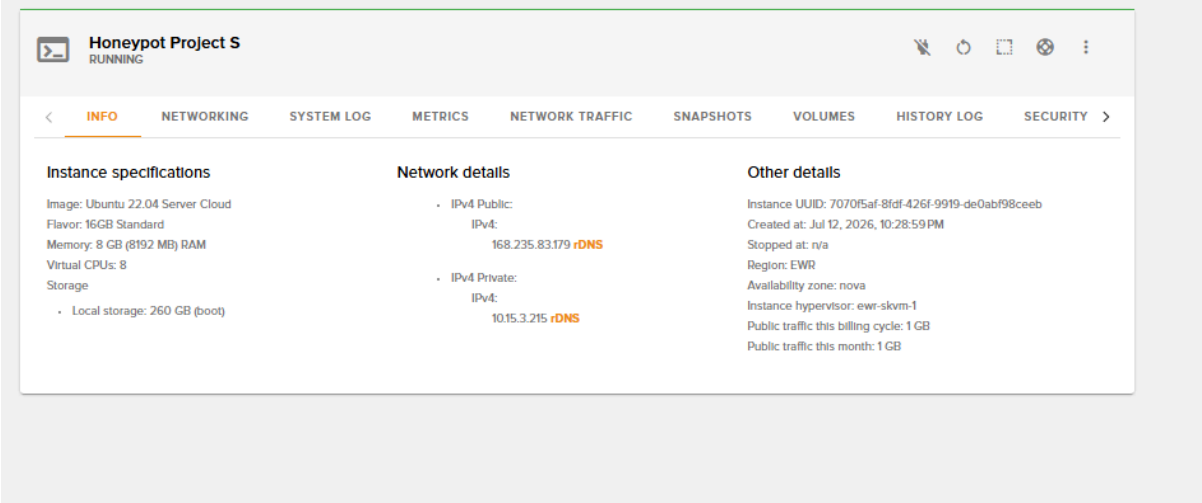
Diagnosing the port 8000 conflict: Splunk's startup check fails, lsof identifies Honeytrap holding the port, and the web UI is relocated.

Honeypot Deployment (T-Pot)

T-Pot 24.04.1 was installed in Hive mode under a dedicated non-root user, deploying the full suite of 20+ containerized honeypot daemons along with its internal Elasticsearch, Logstash, and Kibana stack. Each honeypot runs in its own Docker container, with docker-proxy forwarding the corresponding host ports — SSH, Telnet, RDP, SMTP, SIP, HTTP, industrial protocols — into the right container.

A note on T-Pot's internal ELK stack. T-Pot ships with its own Elastic stack for indexing and dashboards, and running it alongside Splunk meant two full indexing pipelines competing for the same host. Once Splunk was established as the system of record, I stopped the internal Elasticsearch, Kibana, and Logstash containers deliberately — the honeypots kept running independently, and the redundancy was the point of removing them. The same reasoning as dropping a managed service in my previous lab: if another component already demonstrates the capability, the duplicate is just cost.

Even with ELK disabled, the combined footprint of Splunk and 20+ honeypot containers did not hold on 8GB — the box became unresponsive under memory pressure. The practical discovery of this phase was that this workload genuinely requires 16GB. I resized the instance to the 16GB flavor with 8 vCPUs and 260GB of storage, which eliminated the contention entirely.



Instance after resizing to the 16GB flavor — 8 vCPUs and 260GB storage.

Log Ingestion (Splunk Monitors)

With the platform stable, I wired the honeypot log sources into Splunk using the splunk add monitor CLI — one command per source, each tagged with its own tpot:* sourcetype under the main index:

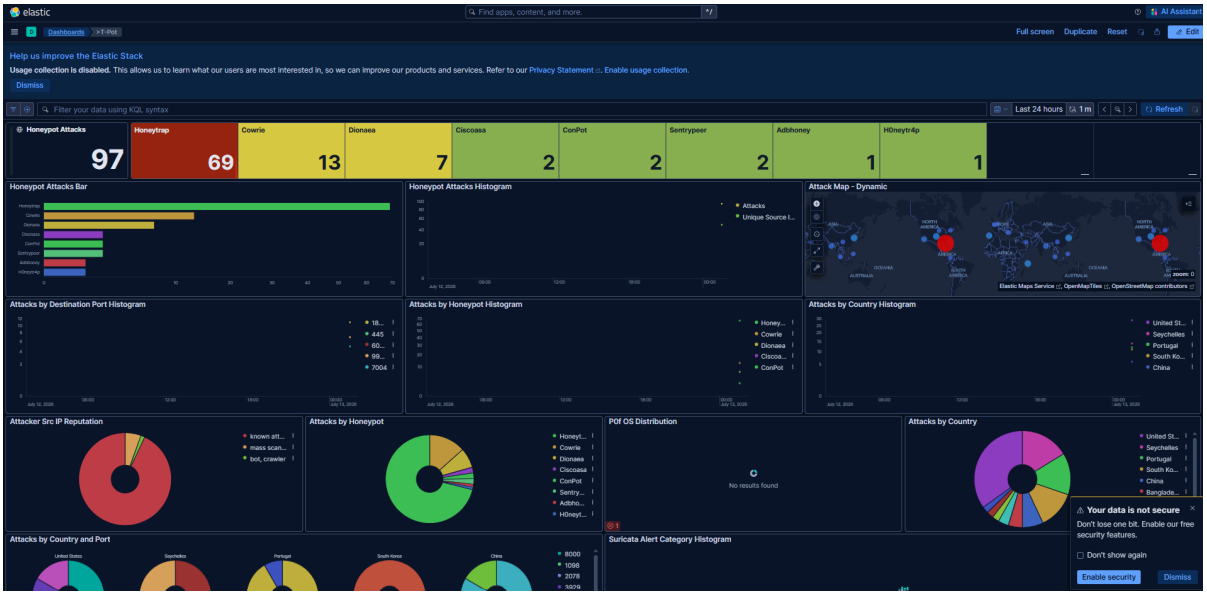
```
sudo /opt/splunk/bin/splunk add monitor /home/tpotuser/tpotce/data/cowrie/log/cowrie.json -index main -sourcetype tpot:cowrie -auth admin:<password>

sudo /opt/splunk/bin/splunk add monitor /home/tpotuser/tpotce/data/honeytrap/log/attackers.json -index main -sourcetype tpot:honeytrap -auth admin:<password>

sudo /opt/splunk/bin/splunk add monitor /home/tpotuser/tpotce/data/dionaea/log/dionaea.json -index main -sourcetype tpot:dionaea -auth admin:<password>

sudo /opt/splunk/bin/splunk add monitor /home/tpotuser/tpotce/data/suricata/log/eve.json -index main -sourcetype tpot:suricata -auth admin:<password>

sudo /opt/splunk/bin/splunk add monitor /home/tpotuser/tpotce/data/conpot/log -index main -sourcetype tpot:conpot -auth admin:<password>
```

The commands were run sequentially — running them in parallel produced conflicts in Splunk's monitor configuration files, while one-at-a-time execution avoided the issue entirely. I then verified data was flowing: a search across the tpot:* sourcetypes returned live events with real attack fields populated, and a stats breakdown confirmed the pipeline scaling as traffic accumulated.

index=main sourcetype=tpot: | stats count by sourcetype — 1,108 events indexed across Conpot, Dionaea, and Honeytrap at time of capture.*

Attack Telemetry

The honeypot began drawing real attack traffic within hours of exposure, without being advertised or linked anywhere — internet-wide scanning finds any exposed IP on common cloud ranges quickly. In the first 24 hours alone, T-Pot recorded 97 attacks originating from the United States, Seychelles, Portugal, South Korea, China, and other countries.



T-Pot's Kibana attack dashboard early in the project — 97 attacks in 24 hours, broken down by honeypot, destination port, and source country.

T-Pot's dashboard after extended runtime — 39k cumulative attacks, led by Honeytrap (19k) and Dionaea (12k), with the histogram showing sustained traffic rather than a single burst.

By the time the instance had been running longer, cumulative attack volume had climbed to 39,000 honeypot events. Honeytrap and Dionaea remained the dominant sources by a wide margin, and the attack map shows source activity concentrated across North America, Europe, and East Asia, with United States, Singapore, Taiwan, and Canada leading the per-country breakdown.

The traffic profile matched what an unadvertised, internet-facing IP typically attracts — overwhelmingly automated, opportunistic scanning rather than targeted intrusion:

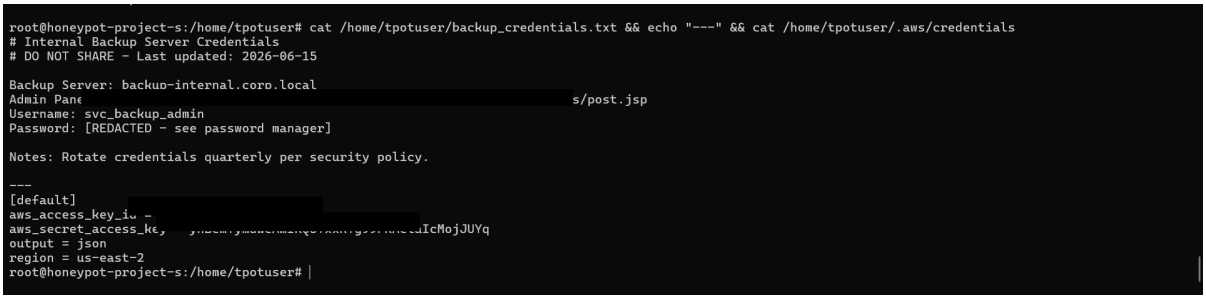
Honeypot	Emulates	What it drew
Honeytrap	Broad catch-all TCP/UDP listener	Highest raw connection count in the early window — the default landing spot for untargeted scans.
Conpot	Industrial / ICS protocols	A steady stream of ICS scanner traffic that came to dominate total event volume.
Cowrie	Fake SSH / Telnet	The more interesting interactive attempts — credential guessing against common username and password pairs.
Dionaea	Malware / exploit capture	Exploit probes and file-transfer attempts against legacy service emulations.
Suricata	Network IDS across all traffic	Signature-level alerting layered over everything the other honeypots absorbed.

Deception Layer (Canarytokens)

The honeypots capture volume — every scanner that touches an exposed port gets logged. To add a high-fidelity signal that fires only on genuine interaction, I planted two honeytokens generated through Canarytokens.org, each placed where real attacker tradecraft would look:

- **Web bug token** — embedded in a fake backup_credentials.txt in the tpotuser home directory, posing as internal backup server credentials with an admin panel URL. Fetching the URL fires the alert.
- **Fake AWS credentials** — a fabricated key pair at the standard ~/.aws/credentials path, the first place an attacker or automated post-exploitation tool checks on a compromised Linux host. Any attempt to use the key fires the alert.

Either token firing sends an email alert directly, independent of the Splunk pipeline — a deliberate choice, so the tripwire keeps working even if SIEM ingestion is down. The token values below are redacted: they are fabricated and connect to nothing real, but publishing them would defeat their purpose as bait and invite false alerts from readers rather than intruders.



Both honeytoken files deployed on the host (token values redacted): the fake backup credentials file and the fake AWS credential pair at ~/.aws/credentials.

LLM Classification (Python + Anthropic API)

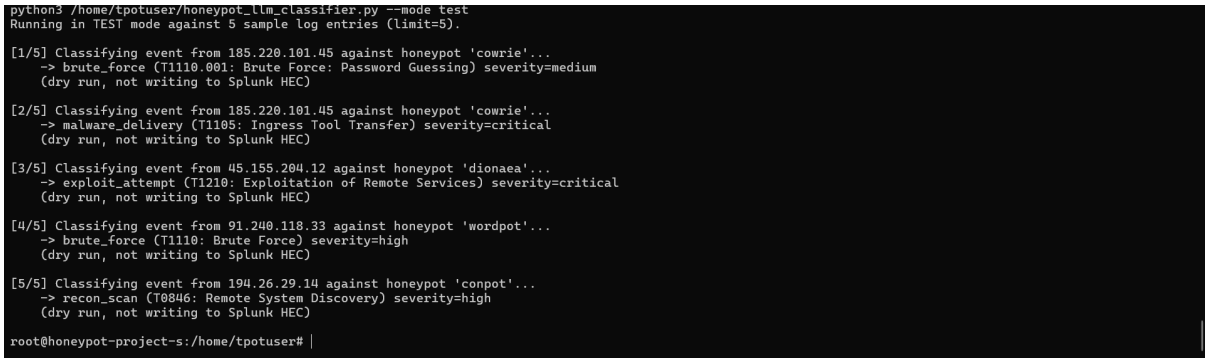
The final stage automates first-pass triage. A Python script, honeypot_llm_classifier.py, pulls the five most recent attack events from Splunk over the REST API, sends each to Claude with a prompt requiring a strict JSON response — attack category, a plain-language summary, the mapped MITRE ATT&CK technique ID and name, and a severity rating — and writes the enriched result back into Splunk through the HTTP Event Collector under the sourcetype tpot:llm_enriched.

The script authenticates twice: to Splunk's REST API for the read path and with a dedicated HEC token for the write path. A --mode flag switches between test (bundled sample events) and live (real Splunk data), and a --write flag toggles dry run versus an actual HEC write — which made it safe to validate the classification logic thoroughly before letting it touch the index.

A note on Gemini. The original plan used Google's Gemini free tier to keep the project zero-cost. It hit a persistent platform-side bug: keys generated through Google's console returned OAuth-style tokens

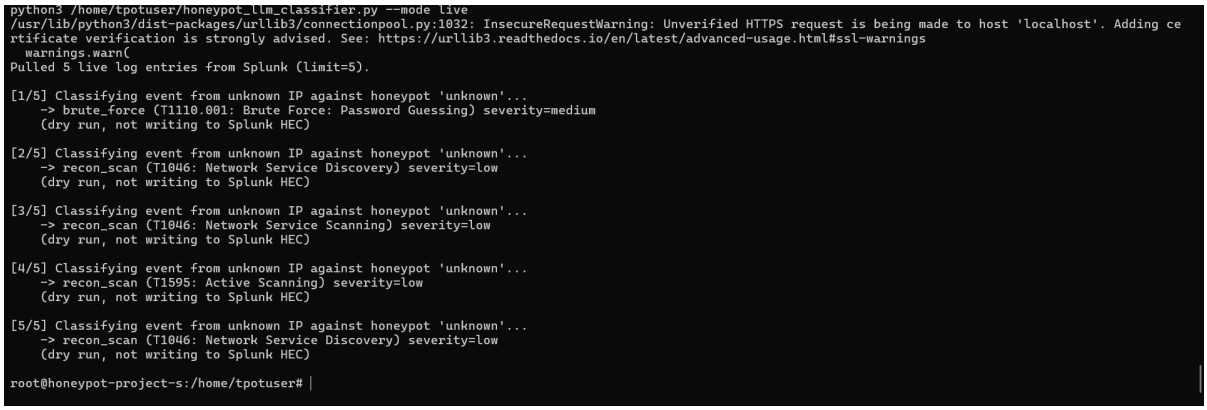
(prefixed AQ.) instead of standard API keys, producing 401 ACCESS_TOKEN_TYPE_UNSUPPORTED and 429 quota errors across both query-parameter and header-based auth. Reproducing the failure across two separate Google accounts ruled out account misconfiguration and confirmed a genuine platform issue — so I switched to the Anthropic API (a small minimum credit purchase), which worked on the first attempt. A defensible root cause is worth more than stubbornly debugging someone else's bug.

I validated the logic first in test mode against five sample log entries. The model correctly distinguished brute-force activity, malware delivery, exploitation attempts, and reconnaissance, mapping each to a specific ATT&CK technique with a severity rating.



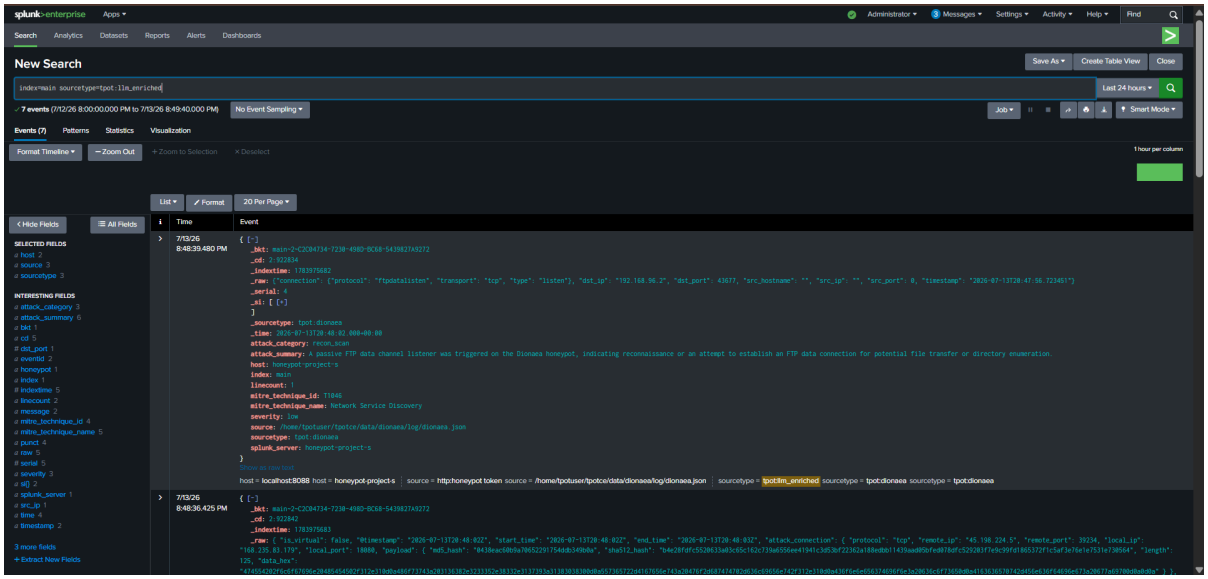
Test mode: five sample events classified — brute force (T1110.001), ingress tool transfer (T1105), exploitation of remote services (T1210), and remote system discovery (T0846).

With the logic validated, I ran the script in live mode against real events pulled from the index. The live window happened to contain a run of scanning activity, which the model classified consistently as reconnaissance at low severity — matching what the underlying honeypot logs showed for that period, and demonstrating sensible severity discrimination rather than uniform alarm.



Live mode: five real events pulled from Splunk and classified as brute-force and reconnaissance activity (T1110.001, T1046, T1595).

Finally, I ran the pipeline with --write enabled and confirmed the loop closed. Searching sourcetype=tpot:llm_enriched returned fully enriched events carrying the original honeypot fields alongside the LLM-generated attack_category, attack_summary, mitre_technique_id, mitre_technique_name, and severity — for example, a passive FTP listener event on Dionaea correctly classified as reconnaissance under T1046, Network Service Discovery.



The loop closed: an enriched event under tpot:llm_enriched showing original Dionaea fields plus the LLM-generated classification, ATT&CK mapping, and severity.

This is the project's thesis realized end to end: raw attacks arrive from the internet, get captured and indexed, and come out the other side as structured, MITRE-mapped, analyst-ready records — the first pass of tier-1 triage, automated.

Pre-Publication Security Review

Because this write-up and its repository are public-facing, I reviewed every artifact for sensitive material before publishing. The honeytoken values were redacted from the deception-layer figure — publishing bait defeats it — and screenshots exposing credentials were cut from the document entirely.

The review also surfaced a real finding: the classifier script contained the Splunk REST credential and HEC token inline. Neither value appears in this document or any published screenshot, but hardcoded credentials in a script destined for a public repository are a genuine exposure. The remediation — moving both secrets into environment variables, rotating the tokens, and adding the config to .gitignore — is being completed before the repository goes public. Finding and fixing this in my own project was exactly the kind of self-audit this lab exists to practice.

Key Decisions & Lessons Learned

- **Size for the real workload.** Splunk plus a full honeypot suite genuinely requires 16GB of RAM — the 8GB starting point was the project's most expensive assumption, and cutting T-Pot's redundant internal ELK stack mattered as much as the resize itself.
- **Co-location is a trade, not a shortcut.** Running the SIEM beside the honeypots traded a networking problem for a resource-management problem — the right trade at this scale, but only after the redundant services were identified and removed.
- **Sequential beats parallel for CLI configuration.** The splunk add monitor commands conflict when run concurrently and succeed cleanly one at a time.
- **A honeypot needs no advertisement.** Internet-wide scanning found this instance within hours of exposure — 97 attacks in the first day.
- **Secrets do not belong in scripts, even in a lab.** Auditing my own artifacts before publication caught hardcoded tokens that would otherwise have shipped to a public repository.
- **Isolate, evidence, escalate.** Both the RamNode outage and the Gemini key bug were resolved by reproducing the failure until the root cause was defensible — then handing it to the party who owned it instead of debugging someone else's platform.

What This Demonstrates

This lab shows I can build and run a detection pipeline against real, uncontrolled attack traffic: deploy and harden internet-facing infrastructure, operate a self-hosted SIEM, wire multi-source log ingestion, layer deception with honeytokens, and automate the enrichment and MITRE ATT&CK mapping that turns raw events into analyst-ready intelligence. Where my earlier labs emulated the attacker, this one waited for the real thing — and processed what showed up the way a working SOC would.

The repository contains the classifier script (with secrets externalized), the Splunk ingestion commands, and the architecture diagram.